

Writing a physics-faithful GNN — the M2 surrogate, element by element

Purpose. An instructional companion to `Phase 5/m2/model_v3.py` and `train_v3.py`: what the network computes, what every element in it *means*, and — the part that generalises — how to write one for a physics target rather than adapting a generic graph network and hoping.

Audience. Anyone (including a future session) who has to modify, extend, port or re-derive the M2 surrogate. It assumes the project's elasticity conventions (`CLAUDE.md` §3, `documentation/MATERIALIZE.md` §3–4) and no machine learning beyond "a neural network is a parameterised function fitted by gradient descent".

Provenance. Implements the model of `Phase 5/m2/M2_V2_PLAN.md` §2.4–2.5 against the solver `Phase 2/forward_solver_torch.py`. Every number quoted here is measured and traceable to `Phase 5/results/m2_s1/ (V3_TENSOR_MESSAGES.md , M2_RESIDUAL_AND_CONSTRAINTS.md )` or to the gates `Phase 5/verifications/test_m2_head_v3.py` and `test_m2_constraints.py`.

What this is not. Not a status report — for where the model currently stands, and whether it is good enough, read `Phase 5/results/m2_s1/M2_RESIDUAL_AND_CONSTRAINTS.md` §6 and `documentation/NEXT_SESSION.md`.

0. The one-paragraph version

The surrogate does **not** predict the number we want. It predicts a small invariant per-triangle correction to a closed form we already have, in a basis that makes rotation-equivariance and positive-definiteness algebraic identities rather than things to be learned, passed along exactly the graphs the solver's own constraint operators couple. Everything below is the reasoning behind each clause of that sentence — and each clause was arrived at by measuring the failure of the version without it.

1. Start from the target, not from the network

The solver computes, per triangle `s`:

$$\begin{aligned} C(s) &= (1 + W)^T A_3(s) (1 + W) & A(s) &= \sum_{e \in s} (k_e / 4\ell_e^2) q_e q_e^T \\ C_{\text{eff}} &= (1/N) \sum_s C(s) & q_e &= \text{vec3}(\Delta x_e \Delta x_e^T) \end{aligned}$$

`A(s)` is a **closed form** in the triangle's own three edge vectors and stiffnesses. `W` is not: it solves a KKT saddle system coupling the whole mesh,

$$\begin{bmatrix} A_3 & C^T \\ C & 0 \end{bmatrix} \begin{bmatrix} W \\ \Lambda \end{bmatrix} = \begin{bmatrix} -A_3 \\ 0 \end{bmatrix}$$

whose constraint block `C` stacks three geometry-only operators (`J_edge` , `C_curv` , `M_S` ; §4).

The structural fact the entire architecture is built on: with *no* constraints the system gives `W = -I`, hence `C(s) = 0`. **The whole elastic response is the constraints.** A network that does not represent them is being

asked to infer, from examples, a global linear solve — and that is exactly what the early versions failed to do.

Two immediate consequences for the design:

- 1. **The easy part must not be learned.** `C` is numerically dominated by `A`: predicting `A` alone scores **1.1747** against the label σ , worse than predicting the global mean (0.9866) — yet the trained free-head model scored 0.2000, so **91 % of the remaining error sat in the correction** while the squared loss was dominated by the term that has a formula. Worse, where $W = 0$ the answer is $A(s)$, and the free head scored **0.0538** there instead of 0. It was spending capacity rediscovering a closed form, imperfectly.
- 2. **The hard part is the constraint structure**, so the constraint structure is what the message graph should be (§4) — not a graph chosen because it is the convenient one.

2. Element 1 — choose what the network outputs

2.1 Predict per-triangle `G`, never `C_eff`

The quantity a designer wants is `C_eff`, six numbers per network. The network predicts the **per-triangle** tensor instead, and `C_eff` is recovered by the project's own homogenisation (the **unweighted** mean; area weighting biases v on unequal-area meshes — `CLAUDE.md` §3). Four reasons, all of which cost something when violated:

reason	what goes wrong otherwise
intensity	v_1 pooled node features with <code>mean + sum</code> . <code>sum</code> is extensive; <code>C_eff</code> is intensive (verified 5e-16–7e-13 across supercells, <code>Phase 5/verifications/supercell_invariance.py</code>). A model with an extensive pooling cannot transfer between sizes even in principle.
supervision density	the same 900-network holdout carries 88 560 per-triangle labels. Predicting six numbers per network throws away four orders of magnitude of signal.
locality	$C(s)$ is a local object with a measurable correlation range; <code>C_eff</code> is not, so a per-network target destroys the very structure a GNN exists to exploit.
downstream use	the edit-policy needs <i>where</i> the response lives, not only its average.

2.2 The head: $C(s) = Q G Q^T$

`Q` is the (3×3) matrix whose **columns** are the triangle's three edge carriers `q_e`. Then

$$C(s) = Q G Q^T$$

with G symmetric PSD

buys two properties **as identities, not as trained behaviour**:

- **equivariance** — under a rotation, $Q \rightarrow SQ$ in the vec3 representation, so $C \rightarrow SCST^T$ automatically, *provided* G is invariant. The network therefore only has to produce an invariant object, which is a far easier learning problem than producing an equivariant one.
- **SPD** — $Q G Q^T$ is PSD whenever G is. A predicted elastic tensor that is not positive semi-definite is not merely inaccurate, it is unphysical, and no loss term can guarantee what an architecture can.

Measured on the gates: rotation invariance of $\mathbf{G}$ at **5.55e-17**, equivariance of $\mathbf{C}$ at **3.84e-16**, min eig($\mathbf{C}$) **1.84e-03** — machine precision, because these are algebra.

2.3 The residual (congruence) head — put the closed form *in* the architecture

$$\mathbf{G} = (\mathbf{I} + \mathbf{X}) \cdot \mathbf{G}_{\text{an}} \cdot (\mathbf{I} + \mathbf{X})^T \quad \mathbf{G}_{\text{an}} = \text{diag}(k_e / 16 \ell_e^2)$$

$\mathbf{G}_{\text{an}}$ is $\mathbf{A}(\mathbf{s})$ expressed in the $\mathbf{Q}$ basis, computed inside `forward()` from inputs already present: ℓ_e^2 is the trace of each carrier ($\mathbf{Q}[:,0,:] + \mathbf{Q}[:,2,:]$, since $\mathbf{q} = [\Delta x^2, \Delta x \Delta y, \Delta y^2]$) and k is the first block of `tri_scalars`. The network predicts only the dimensionless invariant 3×3 $\mathbf{X}$.

Three properties, in the order they matter:

- $\mathbf{X} = \mathbf{0}$ gives **exactly** $\mathbf{A}(\mathbf{s})$ — the correct answer wherever $\mathbf{W} = \mathbf{0}$, and the right *scale* everywhere else;
- the readout's final layer is **zero-initialised**, so training *starts* there rather than at a random tensor of the wrong magnitude;
- a congruence of an SPD diagonal is PSD always, so §2.2's guarantee is inherited.

The general rule, and it is the most portable thing in this document: *if any part of your target has a closed form, put the closed form in the architecture and let the network predict only the residual.* You get exactness on the easy part, the right output scale for free, a physically-meaningful zero initialisation, and a loss that is no longer dominated by the term you did not need to learn.

Caveat, stated because it is easy to overclaim — and now measured. $\mathbf{X} = \mathbf{0}$ reproducing $\mathbf{A}(\mathbf{s})$ is an *initialisation* property. Nothing forces $\mathbf{X} \rightarrow \mathbf{0}$ on $\mathbf{W} = \mathbf{0}$ samples during training, and it drifts: on the 254 holdout networks with $\mathbf{W}$ exactly zero, the trained model has the **lowest per-triangle tensor error of any bin (0.0405) and MAE(v) 0.0528** — above the kill line, on the samples whose answer is a closed form the architecture already contains. A loss term or a hard mask pinning $\mathbf{X} \rightarrow \mathbf{0}$ there is the obvious fix and has not been done. *The lesson generalises: an architectural guarantee that holds at initialisation is not a guarantee about the trained model, and must be re-measured on the checkpoint.*

3. Element 2 — choose the representation (symmetry before layers)

3.1 The hidden state is a pair

$\mathbf{s} : (n_{\text{tri}}, n_s)$ rotation-INVARIANT scalars
 $\mathbf{T} : (n_{\text{tri}}, n_t, 3)$ EQUIVARIANT vec3 tensor channels, $\mathbf{T} \rightarrow \mathcal{S}\mathbf{T}$ under rotation

and the **only** operations used on it are the ones that respect that law:

operation	form	why it is allowed
mix tensor channels	$\mathbf{T}' = \mathbf{W} \mathbf{T}$ ($\mathbf{W}$ acts on the <i>channel</i> index)	scalars commute with $\mathcal{S}$
gate tensors by scalars	$\mathbf{T}' = g(\mathbf{s}) \cdot \mathbf{T}$	same
extract invariants	$\langle \mathbf{T}_i, \mathbf{T}_j \rangle$	a contraction of two equivariant objects
inject geometry	q_e enters messages directly as a tensor	q_e transforms correctly by construction

Anything else — feeding Δx components into an MLP, concatenating a tensor into the scalar path — breaks the symmetry silently and the model then has to spend capacity learning to be rotationally covariant from data augmentation it does not have.

3.2 The invariant inner product is `diag(1, 2, 1)`, not the dot product

`vec3` is `[xx, xy, yy]` with **no factor 2 on shear** (the project's state convention). The rotation-invariant pairing is therefore $\langle A, B \rangle = \text{tr}(AB) = A \cdot \text{diag}(1, 2, 1) \cdot B$, not $A \cdot B$.

Measured: the plain dot product changes by up to **11×** under rotation; `diag(1, 2, 1)` is invariant to **1.1e-14**. This is the single easiest way to silently destroy the equivariance you just built, and it has bitten this project in three separate places (the state-vs-constraint factor of 2 also makes a raw comparison of the model's bond carriers against the solver's `J_edge` rows show a spurious relative error of 0.5 — see `mesh_build.kkt_from_tri_bond`).

Rule. Write the invariant pairing as a named constant (`VEC3_METRIC`) used by *one* function (`inner`), and make a rotation gate test it numerically. Never inline it.

3.3 Why scalar-message GNNs fail here — measured, not asserted

The predecessor (`model_v2.py`) reduced each bond to a scalar length and passed scalar messages between **nodes**. Consequences, measured:

- where $W = 0$, it learned $C(s) = A(s)$ to 0.001 with **zero** message-passing layers — the easy part needs no graph at all;
- where $W \neq 0$, triangles whose scalar features were as close as the data allows had targets differing by **0.716** (MAE/ σ), against 0.697 for predicting the global mean. Local scalar proximity bought essentially nothing;
- depth did not help: 0, 1 and 2 layers all landed at ~ 0.51 — the "predict your own bulk" baseline.

The information that decides W is the **relative arrangement** of adjacent triangles, and scalar messages destroy it twice: once by collapsing each bond to a length, and once by aggregating between nodes when the physics couples triangles. Passing tensors on triangle adjacency moved the same holdout from 0.5111 to 0.3906.

4. Element 3 — the graph is the physics' coupling, not a convenience

This is the step most easily got wrong, because *any* graph "works" in the sense of running. The solver's constraint block tells you which one is right. All three operators are **geometry-only** (`forward_solver_torch.py:613`), i.e. they do not depend on `k` — which is why they can be baked into the architecture at all.

operator	rows	what it says	the graph it induces	module
<code>J_edge</code>	1.5 N	the two triangles sharing a bond must agree on that bond's length	triangle adjacency through shared bonds	<code>TensorMP</code>
<code>C_curv</code>	0.5 N	zero discrete Gaussian curvature: $\sum_{\{s \ni v\}} (\partial \theta_{v^s} / \partial g) \cdot \delta g(s) = 0$	the vertex star — all ~6 triangles at a vertex, <i>simultaneously</i>	<code>StarMP</code>
<code>M_S</code>	3	the area-weighted mean of the metric field vanishes	a global , rank-3 coupling	<code>GlobalMS</code>

4.1 `J_edge` — verify what you think you already have

`triangle_adjacency` (two triangles per shared bond) **is** the edge-compatibility pairing. That had been asserted in this project for a long time and never checked; when it finally was: pairing identical (177 solver edges → 354 directed, exact set match) and weights identical **once contracted with** `diag(1,2,1)` — `max rel = 0.000e+00`.

Rule. "The architecture already has this coupling" is a claim, and claims get gates. Half of this section's content came from testing two assumptions, one of which held exactly and one of which turned out to be entirely absent.

4.2 `C_curv` — the channel that was missing, and the largest single gain

`TensorMP` cannot express a vertex star: it is a *pairwise* coupling through bonds, and curvature couples all triangles at a vertex at once. `StarMP` is a bipartite triangle → vertex → triangle round that does.

Two details worth copying:

- **The constraint's own weight is passed, not a proxy.** `∂θ/∂g` is computed by `model_v3.angle_gradient_vec`, a **literal port** of the solver's `_angle_gradient_vec` — verified elementwise at **0.000e+00** and as an assembled operator at 4.4e-16 / 8.3e-16 / 0.0 on three meshes. It is not an approximation of the solver's convention; it is it, including the sign convention for which edge vector is taken at which vertex.
- **A vec3 weight is split into direction and magnitude:** a unit tensor `w/|w|` (equivariant) plus `log1p(|w|)` (invariant). Passing `w` raw would let its scale — which spans orders of magnitude on sliver corners — swamp the tensor channels, and the normalisation would then have to undo it.

Result: 0.2023 → 0.1209 per-triangle MAE/σ, -40 %, improving in every k-contrast bin.

4.3 `M_S` — cheap to represent exactly, but it costs the receptive field

The quantity the constraint zeroes is the area-weighted mean of the metric field, so handing the model that mean is the constraint's whole content, not a proxy for it. Rank 3, and per-triangle influence falls as `1/N`.

The cost is stated, not hidden: with `M_S` on, one bond's perturbation reaches **112/112** triangles in a single layer. The model is no longer finite-reach. That is *faithful* — the constraint really is global — but it compounds the global normalisers already in `prepare()`, and it is the prime suspect for the measured size-transfer degradation. Hence `--no_global`, so it can be ablated as a single variable.

4.4 The check that the constraint set is the right one

$3N - \text{rank}(\text{stacked constraints})$ should equal the dimension of the space of metric fields some node configuration could actually produce, $2 \cdot \text{nodes} - 2$:

mesh	3N	J_edge	C_curv	M_S	rank	3N – rank	2·nodes – 2
random_patch(60)	354	177	59	3	238	116	116
random_patch(120)	696	348	116	3	466	230	230
bravais r4	96	48	16	3	66	30	30

Exact on all three, and the stacked operator is rank-deficient by exactly 1 — the redundancy `CLAUDE.md` records as the root of the B-1 mis-solve, rediscovered here independently.

4.5 Reach arithmetic — count it, do not guess it

Each layer runs `TensorMP` (one **bond** hop) then `StarMP` (one **star** hop), so a layer advances **two** hops on the union graph, not one. This was got wrong twice while writing the gate, and a wrong bound fails a correct model. With `M_S` off, the gate confirms exactly: `L=1` → nothing beyond 2 union hops, `L=2` → nothing beyond 4.

5. Element 4 — normalise so that the target is reachable at all

Three normalisations, each answering a specific measured failure.

Length and stiffness scale. `q_e` has dimension length^2 , so an unnormalised `Q` makes the predicted `C` scale as ℓ^2 — while the target is scale-**invariant** and every input feature is too. The head would have to guess a per-network factor it cannot see. Measured spread of ℓ^2 across the dataset: 0.752 to 1.600, a factor ~ 4 in `C`. Fix: build `Q` from $\Delta x / \ell^2$ and $k / \bar{k}$, and move ℓ^2 into the physical factor, where $\text{phys} \cdot \ell^2$ is itself scale-invariant.

Tensor magnitude (`tensor_rms_norm`). The readout consumes (T, T) , which is *quadratic* in `T`: any growth through the residual updates squares into the readout. Unnormalised, the first training step saw a loss of **2e33**. The fix must divide by an **invariant scalar built from `T`'s own invariants** — a scalar commutes with $T \rightarrow \delta T$, so stability costs no equivariance. (Dividing by anything non-invariant would have broken §3.)

Scalars get `LayerNorm`; tensors never do. `LayerNorm` mixes and shifts components, which is meaningless — and symmetry-breaking — on a `vec3`.

Pool per graph, never across the batch. `collate` packs graphs block-diagonally, so a plain `T.mean(0)` in `GlobalMS` would average across unrelated networks. `tri_batch` exists for that. This is not hypothetical: `prepare()` once set `area_w` but not `tri_batch`, and since `forward` enables `M_S` on `'tri_batch'` in `g`, **single-sample evaluation silently ran a different architecture than batched training** — 7.8e-02 apart, now 0.000e+00. A single sample is a batch of one and must say so.

6. Element 5 — loss, schedule, filter: every knob is a scientific choice

`CLAUDE.md` §3 "Learned-model choices: NO UNJUSTIFIED DEFAULTS" is the governing rule, and it exists because two framework defaults each produced a wrong claim. State, for every knob: **what it does, why it suits this target, and what it would fake.**

knob	choice	what it would fake
loss	squared error on the σ - normalised residual; <code>--huber 1.0</code> on top	the label distribution is heavy-tailed, so a few triangles dominate the squared gradient. Huber bounds their influence — that is robustness , not regularisation ; the model underfits, so weight decay would push the wrong way.
schedule	<code>ReduceLROnPlateau</code> + early stopping	<code>CosineAnnealingLR</code> manufactures convergence : the LR reaches ~ 0 at <code>T_max</code> , so the curve flattens by construction. At epoch 200/220 the rate was 2 % of initial, at 219 it was 0.005 %, and the flat tail 0.3900/0.3920/0.3900/0.3902 was reported as "converged". A clock-driven schedule cannot distinguish <i>found the bottom</i> from <i>no longer allowed to walk</i> .
stopping	<code>min_delta = 2 %</code> , measured against noise	the previous 0.2 % sat $\sim 40\times$ below the evaluation-to-evaluation scatter ($\sim 9\%$ early, $\sim 0.5\%$ late), so noise read as progress and the run never stopped.
optimiser	Adam, <code>wd = 0</code>	Adam and AdamW are identical at <code>wd = 0</code> , so the flag is really the question "should we use weight decay". With <code>LayerNorm</code> / <code>tensor_rms_norm</code> downstream, decay on a pre-normalisation weight leaves the output unchanged and acts as an <i>effective-LR modifier</i> , not regularisation — separate those before calling a gain regularisation.
<code>--w_max_cut 10</code>	drop near-mechanism training samples (34.6 % of them); never filter the holdout	this is a domain restriction , and it must stay visible in scoring. Filtering the holdout too would make the model look better by deleting the cases it cannot do.

The decisive convergence test is an LR-restart probe, not the shape of a curve: take the best checkpoint, restore the initial LR, train on. If the score improves, the flat tail was the schedule.

Two more habits the runs paid for:

- The checkpoint name must encode the ARCHITECTURE, not only the data flags. A tag keyed on holdout+filter+layers made the residual-head run byte-identical to the free-head run before it — it would have hit the no-overwrite guard and discarded ~ 24 h. `HEAD_VERSION`, `_star`, `_ms` and the epoch count are all part of the identity now.
- Snapshot at every evaluation (`--resume`, atomic via `.tmp` + `os.replace`). A 50-hour run with no snapshots costs 50 hours to a power cut, and did.

7. Element 6 — gate the architecture *before* you train it

Every architectural property claimed above is a test in `Phase 5/verifications/test_m2_head_v3.py` and `test_m2_constraints.py`. Running them takes seconds; the runs they protect take days.

gate	what it would catch
<code>G</code> invariant / <code>C</code> equivariant under rotation	a broken inner product, a tensor leaked into the scalar path
<code>min eig(C) > 0</code>	a head that can emit non-physical tensors
untrained head == analytic <code>A(s)</code> (rel 0)	the residual head no longer starting at the closed form
a nonzero <code>X</code> moves <code>G</code>	a dead correction channel — the model would look stable and learn nothing
receptive field: nothing moves beyond <code>2L</code> union hops	an accidental global coupling
with <code>M_S</code> on, <code>L=1</code> reaches ~all triangles	the global channel silently off
batched vs one-at-a-time output identical	the train/eval architecture split of \$5
model's <code>J_edge</code> / <code>C_curv</code> vs the solver's own operators	a convention drift between model and solver

Three real bugs these caught, all of which would have been read as model quality:

1. the global channel silently off during evaluation (§5);
2. **the zero-init made the receptive-field gate vacuous** — with the readout zeroed, `X ≡ 0` and `G = G_an` depends only on each triangle's own edges, so the influence counts collapsed to 2/6 and 2/13. The gate now randomises the readout first. *A test can be invalidated by an unrelated, correct architectural change*;
3. the reach bound, wrong twice (§4.5) — the test failed the model for the test's error.

8. Element 7 — score against something the model did not train on

Three distinct comparisons; do not collapse them.

- **vs the solver labels** — what the trainer reports. Necessary, never sufficient: it measures agreement with the thing that generated the labels.
- **vs the independent sim** — the headline (`evaluate_v2.py`). `MAE(v) ≤ 0.02` and `MAE(E)/E ≤ 5 %` is the must-tier; `MAE(v) > 0.05` is the kill criterion. Always report **SOLVER vs SIM** on the same networks alongside: it is the floor, because no model trained on solver labels can beat it. (Currently 3.6e-08 — the floor is not the problem.)
- **vs the baselines that make "small MAE" meaningful** — the global mean, and above all the **own-bulk oracle**: predict each network's own bulk `C_eff` at every triangle. That oracle is *handed the answer's average* and asked only for the fluctuation. Merely matching it means the model learned the bulk response and nothing about the per-triangle field — which is exactly what v2 did (0.5111 vs the oracle's 0.4114).

And stratify before averaging. The holdout's difficulty varies by more than an order of magnitude, so the headline moves for reasons unrelated to the change being tested (`Phase 5/verifications/m2_error_strata.py`):

- by **k-contrast**, the error was monotone and 17× end-to-end on the free head. `THEORY_NOTES.md` independently measured the required cluster radius as ~1 for geometric disorder but **4–6 at 100× contrast** — the physics has a *contrast-dependent* screening length while a fixed-depth GNN has one fixed reach. That is a structural mismatch, not a training problem.

- by **max||W||**, which is the *training-domain* boundary (`--w_max_cut`). Reporting the two sides separately is the difference between "the model is bad" and "the model is being scored outside the domain it was fitted on".
- counter-intuitively, the **k-field's own correlation length works the opposite way**: near-white `xi_short` is worst (0.2816), long-wavelength `xi_long` best (0.1582). Long-wavelength stiffness is locally smooth and therefore *easy*.

9. Recipe — adding a new channel

The three existing channels were built this way; a fourth should be too.

1. **Name the physical coupling.** Which term in the governing equations couples which objects? If you cannot write it as an operator with rows, you do not yet know what channel to build.
2. **Write down the incidence it induces**, and build it in NumPy in `prepare()` — the graph is data, not a layer. Give indices their **own offsets** in `collate` (vertex indices are not triangle indices; reusing the triangle offset silently fuses stars across graphs).
3. **Port the operator's weights literally** from the solver, and gate them elementwise against it. Do not re-derive: a re-derivation is a second convention to keep in sync.
4. **Classify each weight as invariant or equivariant**, and route it accordingly — magnitude into the scalar path (usually as `log1p`), direction as a unit tensor into the tensor path.
5. **Write the module** using only §3.1's four operations. Residual update, then normalise: `LayerNorm` for scalars, `tensor_rms_norm` for tensors.
6. **Add a flag to disable it** (`--no_star`, `--no_global`) *before* the first run, and put it in `run_tag`. A channel you cannot ablate is a channel whose effect you cannot measure — and two channels enabled at once is two variables.
7. **Gate it**: does it move the output at all? does it reach exactly the triangles it should? does the batched result equal the one-at-a-time result?
8. **Run it as a single variable** against the previous best, on the same data, split and seed, and **stratify** the result (§8).

`run_tag` is the run's **IDENTITY, not a label**. It names both the checkpoint and the `.resume` snapshot, and `--resume` matches by that name alone — so anything that changes what the run *is* belongs in it: architecture, epochs, objective flags, optimiser, **and the dataset** (`_dv2s0`, `_dbase` ; added 2026-09-12). Two incidents, both from something missing: a 2-epoch benchmark overwrote a 220-epoch checkpoint (11.5 h lost, junk committed in its place), and a resume against the wrong `--data` silently continued a trajectory on a *different* sample draw, since `--overfit_probe n` picks its `n` by `rng(seed).choice(len(raw))`. Recording a value inside the saved `.pt` makes a finished run traceable; only the tag prevents the collision.

10. Element glossary

Files

file	role
<code>Phase 5/m2/model_v3.py</code>	the model: tensor algebra, the three channels, the residual head, <code>from_checkpoint</code>
<code>Phase 5/m2/model_v2.py</code>	the scalar-message predecessor — retained as the reference the tensor version is measured against; also the home of <code>edge_carriers</code> , <code>assemble</code> , <code>sym3_to_c6</code> , <code>vec3_rotation</code> , which v3 imports rather than duplicates
<code>Phase 5/m2/train_v3.py</code>	<code>prepare</code> (sample $\rightarrow$ tensors), <code>collate</code> (block-diagonal batch), the loop, <code>run_tag</code> , <code>oracle_check</code>
<code>Phase 5/m2/build_dataset.py</code>	the labelled data: families, geometry variants, <code>k</code> -patterns, correlation lengths
<code>Phase 5/m2/evaluate_v2.py</code>	the headline score vs the independent sim
<code>Phase 5/verifications/m2_v3_report.py</code>	vs the baselines, on the same split, plus the train-split diagnostic
<code>Phase 5/verifications/m2_error_strata.py</code>	the stratifications of §8
<code>Phase 5/verifications/test_m2_head_v3.py</code> , <code>test_m2_constraints.py</code>	the gates of §7

Tensors that flow through `forward`

name	shape	meaning
<code>Q</code>	<code>(n_tri, 3, 3)</code>	the triangle's three edge carriers <code>q_e</code> as columns (so $Q \text{diag}(c) Q^T = \sum c_e q_e q_e^T$), lengths normalised by ℓ^-
<code>tri_scalars</code>	<code>(n_tri, 9)</code>	per-edge $k/\bar{k}$, ℓ_o/ℓ^- , $\log(k/\bar{k})$ — three each
<code>tri_src</code> , <code>tri_dst</code>	<code>(E,)</code>	directed triangle adjacency through shared bonds (<code>J_edge</code>)
<code>bond_q</code> , <code>bond_feat</code>	<code>(E, 3)</code> , <code>(E, 2)</code>	the shared bond's carrier, and its <code>(k, \ell_o)</code>
<code>star_tri</code> , <code>star_vert</code>	<code>(P,)</code>	the (triangle, corner) incidence of the vertex stars (<code>C_curv</code>)
<code>star_w</code>	<code>(P, 3)</code>	$\partial\theta/\partial g$ at that corner, a vec3
<code>area_w</code> , <code>tri_batch</code>	<code>(n_tri,)</code>	area weights normalised per graph, and each triangle's graph id (<code>M_S</code>)
<code>s</code> , <code>T</code>	<code>(n_tri, ns)</code> , <code>(n_tri, nt, 3)</code>	the hidden state: invariant scalars, equivariant tensors
<code>G</code>	<code>(n_tri, 3, 3)</code>	the invariant output; $C(s) = Q G Q^T$

Symbols

N triangles · $\mathbf{q}_e = \text{vec3}(\Delta \mathbf{x} \Delta \mathbf{x}^T)$ edge carrier · $\mathbf{A}(s)$ bare per-triangle tensor · $\mathbf{W}$ strain-concentration operator · $\boldsymbol{\Lambda}$ KKT multipliers · $\Delta \mathbf{g}$ macroscopic strain (a vec3) · $\mathcal{S}$ the vec3 representation of a rotation · n_s / n_t scalar/tensor channel counts.

11. What this architecture still does not have

Honest limits, so nobody re-derives them:

- **the constraints are represented, not imposed.** The channels carry the operators' *structure and weights*; nothing in the model solves the KKT system or projects onto its null space. Whether representation is enough is the open question the current runs are measuring.
- **one fixed reach per sample**, against a screening length that grows with k-contrast (§8). Depth is one answer; a contrast-adaptive mechanism would be another, and neither has been tested.
- **M_S makes the model globally coupled** (§4.3), which is faithful but interacts with the global normalisers and with size transfer.
- **$\ell_o = \ell$ in all current data**, so that input channel carries no information yet. It becomes an independent design channel under the residual-stress programme, and the model already takes it.
- **the head is $\mathbf{C}(s)$, not $\mathbf{W}$.** Predicting $\mathbf{W}$ directly (and contracting it with the exact $\mathbf{A}_3$) would isolate the hard part completely. It has not been tried.